PYDANTIC

Problems:

1. Type Validation
2. Data Validation
3. Excess Manual/Boilerplate Code Writing

How does Pydantic work?

1. Define a Pydantic model(class) - that represents the ideal schema of the data
   Includes the expected fields, data types, and any validation constraints
2. Instantiate the model with raw IP data (creating a Pydantic object)
   Pydantic will automatically validate the data
3. Pass the validated model object to functions or use it in the codebase

```
def insert_patient_data(name, age):

    print(name)
    print(age)
    print('inserted')
```

Install Pydantic v2 - built on RUST

from pydantic import BaseModel
BaseModel is used to check whether the incoming data is valid or not

class name(BaseModel): → Any class inheriting BaseModel is a Pydantic Model
Define ideal schema

Image

Define a raw dictionary [patient_info = {' ' : ' ' , ' ' : ' '}
Create a Pydantic object [pateint1 = dictionary(**dictonaryname) —> to unpack the dictionary]

When you run →
The rules will be applied; if everything works fine, we get the Pydantic object

```
def insert_patient_data(patient: Patient):

    print(patient.name)
    print(patient.age)
    print('inserted')
```

Now, instead of using name and age inside the function, we can use the Pydantic object directly. Now we call the function [insert_patient_data (patient1)]

Apart from this, Pydantic is smart enough —> str format is converted into int automatically

To mention a list of string, we have to import List from the typing module

Allergies: List[str]
Only list is not used to enable two-level validation
Allergies is a list, but inside it, every item is a string

Import Dict from the typing module
Contact_details: Dict[str,str] —> key,value
Same for a dict

To add the details, we add them to the patient_info raw dictionary
patient_info = {'name':'nitish', 'email':'abc@gmail.com', 'linkedin_url': 'http://linkedin.com/1322', 'age': '30', 'weight': 75.2, 'contact_details':{'phone':'2353462'}}

Import Optional from pydantic
Allergies: Optional[List[str]] = None
Married: bool = False
Whenever we use Optional for any field, we have to give a default value

DATA VALIDATION

1. Pydantic gives custom data types for data validation other than Python data types
   Import EmailStr from the pydantic module
   Email: EmailStr
   Now, whenever the email is provided, it will be validated against the custom data type - For Valid Email String Format
   Import AnyUrl from the pydantic module
   Linkedin_url: AnyUrl

2. Custom Data Validation - Field Function [can be used for both numerical & string data types]

Weight shouldn't be negative, so we apply a constraint

```
Weight: float = Field(gt=0)
```

Range for age field

```
Age: int = Field(gt=0, lt=120)
```

For string-based data type:

```
Name: str = Field(max_length=50)
Allergies: Optional[List[str]] = Field(max_length=5)
```

Field Function purpose is not only for data validation, but it is also used to attach metadata (description)

Eg: Building APIs in documentation (FASTAPI)

Annotated from the typing module

Annotated[datatype, constraints, metadata]

Annotated from the typing module + Field from the Pydantic module

```
Name: str = Annotated[str, Field(max_length=50, title='Name of the patient', description= 'Give the name of the patient in less than 50 chars', examples=['Priya', 'Sonu'])]
```

Can set a default value too

```
Married: Annotated[bool, Field(default = None, description='Is the patient married or not')]
Allergies: Annotated[Optional[List[str]] = Field(default=None, max_length=5)]
```

Suppress type coercion behavior using Field Function

```
Weight: Annotated[float, Field(gt=0, strict=True)]
```

Field Validator

A given EmailStr contains hdfc.com/icici.com checking is also data validation, but the problem is that it is a specific business use case data validation; we use a Field Validator.

In these fields, we can do custom data validation + transformation too
Like the patient's name should always be capitalized - A kind of transformation

1. Data Validation
Import Field Validator from Pydantic
Have to create a method - make sure it is used along with the decorator

```
@fieldvalidator('email')
@classmethod                    → have to mention a classmethod
def email_validator(cls, value):      → value is the one the patient provides; cls is Patient - if there are any other methods we can access here, hence we mentioned cls(adv)
        valid_domains = ['hdfc.com', 'icici.com']
# now we are extracting the domain name - abc@gmail.com
        domain_name = value.split('@')[-1]          → after splitting, we get abc as one list and gmail.com as the other list; as we want only the domain, we use the -1 index
        if domain_name not in valid_domains:
                Raise ValueError('Not a valid domain')
        return value
```

2. Transformation

```
@fieldvalidator('name')
@classmethod                    → have to mention a classmethod
def transform_name(cls, value):
        return value.upper()
```

Field Validator operates in 2 modes: Before Mode & After Mode

```
        @fieldvalidator('name', mode='after')       → after mode is by default
```

We will get the value after the type coercion

```
@fieldvalidator('age', mode='before)
@classmethod
def validate_age(cls, value):
        if 0 < value > 100:
                return value
```

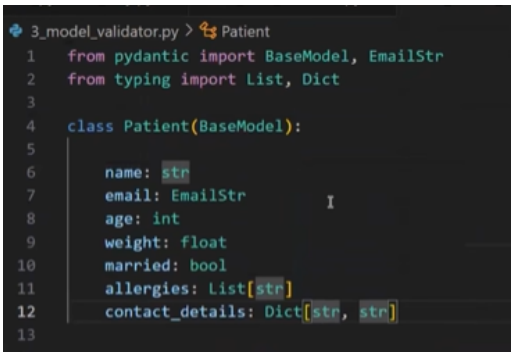
```
        else:
            raise ValueError('Age should be in btw 0 and 100')
patient_info = {'name':'nitish', 'email':'abc@gmail.com', 'linkedin_url':
'http://linkedin.com/1322', 'age': '30', 'weight': 75.2,
'contact_details':{'phone':'2353462'}}
```

Here age will be string only as we mentioned mode='before' → so give value before type coercion

O/P: Error - saying can't compare str with int

Hence we mention and by default is mode='after' → gives value after type coercion

Model Validator



Used to apply data validation on multiple fields and will be inter dependent

Import modelValidator from pydantic

@modelvalidator(mode='after') → No need to mention any specific field as we are now working on complete pydantic model

def validate_emergency_contatct(cls,model): → As we are now working on complete pydantic model that is we can extract any of the field value so we use model instead of just value

```
        if model.age > 60 and 'emergency' not in model.contact_details:
            raise ValueError('Patients older than 60 must have an emergency contact')
        return model

patient_info = {'name':'nitish', 'email':'abc@gmail.com', 'linkedin_url':
'http://linkedin.com/1322', 'age': '70', 'weight': 75.2,
'contact_details':{'phone':'2353462'}}
```

O/P: Value Error, Patients older than 60 must have an emergency contact

```
patient_info = {'name':'nitish', 'email':'abc@gmail.com', 'linkedin_url':
'http://linkedin.com/1322', 'age': '70', 'weight': 75.2,
'contact_details':{'phone':'2353462', 'emergency': '237489550556'}}
```

O/P: No Error

Computed Fields

The field value is not provided by user, infact using the other field values we calculate the computed field value

Using the weight and height we calculate the BMI dynamically. So this BMI field is Computed field

Import computed_field from pydantic

```
@computed_field
@property
def bmi(self) -> float:
        bmi = round(self.weight/(self.height**2),2)
        return bmi
```

```
patient_info = {'name':'nitish', 'email':'abc@icici.com', 'age': '65', 'weight': 75.2,
'height': 1.72, 'married': True, 'allergies': ['pollen', 'dust'],
'contact_details':{'phone':'2353462', 'emergency':'235236'}}
```

Nested Models

Using one pydantic model inside another pydantic model as a field

Import BaseModel from pydantic

address: 'house no 2, sec 66, gurgaon, 122002'
Address is a complex data type, ideally in this situation we have to create another pydantic model for address so that this can be used as a field in Patient model —> Nested Model

```
class Address(BaseModel):

    city: str
    state: str
    pin: str

class Patient(BaseModel):

    name: str
    gender: str
    age: int
    address: Address
```

```
address_dict = {'city': 'gurgaon', 'state': 'haryana', 'pin': '122001'}
# Address pydantic model object creation
address1 = Address(**address_dict)        → unpapcking the address_dict

patient_dict = {'name': 'nitish', 'gender': 'male', 'age': 35, 'address': address1}
# Patient pydantic model object creation
patient1 = Patient(**patient_dict)          → unpapcking the patient_dict

print(patient1)
print(patient1.name)
print(patient1.addresspin)

# Better organization of related data (e.g., vitals, address, insurance)
# Reusability: Use Vitals in multiple models (e.g., Patient, MedicalRecord)
# Readability: Easier for developers and API consumers to understand
# Validation: Nested models are validated automatically—no extra work needed
```

Exporting the pydantic models objects as python dictonary/json
Python gives built-in methods

```
temp = patient1.model_dump()       → model.dump() converts pydantic models object into
python dict
print(temp)
print(type(temp))


temp = patient1.model_dump_json()         → model.dump() converts pydantic models object
into json
print(temp)
print(type(temp))              → Python will receive it as str


We can control which values to be exported - include, exclude, exclude_unset
temp = patient1.model_dump(include=['name', 'gender'])
temp = patient1.model_dump(exclude=['address':['state'])
```

If there is a default value for gender: 'male'

```
address_dict = {'city': 'gurgaon', 'state': 'haryana', 'pin': '122001'}
address1 = Address(**address_dict)

patient_dict = {'name': 'nitish', 'age': 35, 'address': address1}
patient1 = Patient(**patient_dict)

temp = patient1.model_dump(exclude_unset=True)
```

As there is no gender in the patient_dict pydantic model object , gender will not be exported even though there is a default value

```
PS C:\Users\Nitish\Desktop\pydantic-tutorial> & C:/Users/Nitish/AppData/Local/Program
thon/Python311/python.exe c:/Users/Nitish/Desktop/pydantic-tutorial/6_serialization.p
{'name': 'nitish', 'age': 35, 'address': {'city': 'gurgaon', 'state': 'haryana', 'pin
122001'}}
<class 'dict'>
PS C:\Users\Nitish\Desktop\pydantic-tutorial>
```

Bcs it was not set during the object creation process